

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет безопасности информационных технологий

Направление подготовки: 10.03.01 Информационная безопасность

Образовательная программа: Информационная безопасность

Дисциплина:
«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
«Реализация сервиса для взаимодействия с разработанной базой данных»

Выполнил студент(ы):

группа/поток: N3347 / ИББД.Н63 1.5



Чу Ван Доан /

ФИО

Подпись

Проверил:

Салихов Максим Русланович /

ФИО

Подпись

Отметка о выполнении (один из вариантов:
отлично, хорошо, удовлетворительно, зачтено)

Дата

Санкт-Петербург
2024 г.

СОДЕРЖАНИЕ

Цель работы.....	2
Задание.....	4
Ход Работы.....	5
1. Описание лаборатории.....	5
1.1 Структура проекта.....	5
1.2 Описание работы.....	5
2. Выполнение программы.....	7
3. Основные моменты в разработке.....	10
3.1 Основной исходный код.....	10
3.1 Маршруты (функциональные модули).....	13
4. Анализ характеристик приложения.....	19
Вывод.....	20

Цель работы

Получение навыков использования серверных языков программирования, фреймворков по работе с БД, ORM-систем

Задание

1. На основе БД, созданной в предыдущих лабораторных работах, необходимо создать сервис, который взаимодействует с разработанной БД. В качестве сервиса среди прочего могут быть разработаны:

- веб-интерфейс;
- desktop приложение;
- API;
- сервисы предобработки данных, которые заполняют базу данных (обработка websocket соединений, парсинг страниц, json и др)
- сервисы для защиты данных при их обработке, записи, считывании из/в БД (шифрование, обезличивание, сетевая защита и др.) (список не исчерпывает все возможные сервисы, см. п.4 в примечании)

Дайте краткое обоснование выбранной среде разработки, языку и фреймворку, который вы используете для взаимодействия с базой данных.

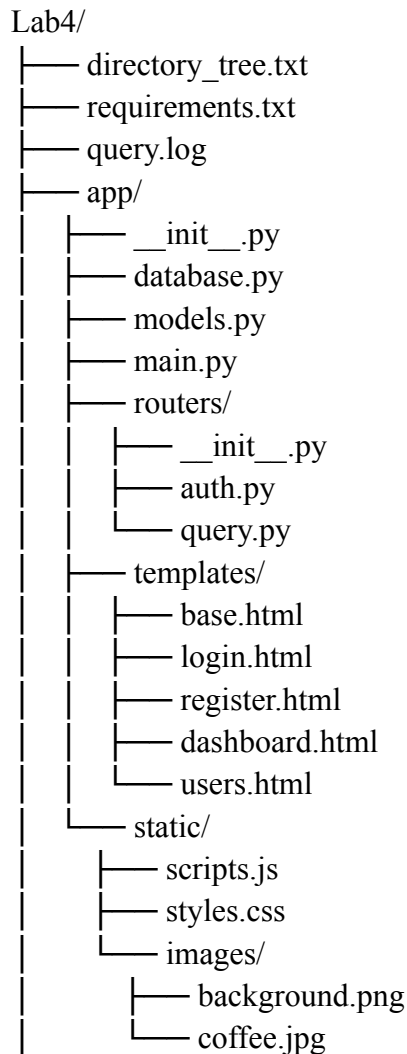
2. В рамках ЛР опишите каким образом вы взаимодействуете с базой данных. С помощью каких фреймворков или языков. Продемонстрируйте несколько примеров по выборке, вставке, удалении данных из вашей БД с помощью выбранного вами фреймворка или языка программирования. Проконтролируйте корректность работы систем логирования и разграничения доступа, которые реализованы в 1-3 ЛР.

3. Для реализованного сервиса укажите кратко его основные функции, назначение, кратко опишите структуру. Дополнительно опишите и оцените методы защиты данных, реализованные вами и имеющиеся в наличии в разработанном сервисе. Выделите аналоги, недостатки и преимущества решений, которые вы реализовали.

Ход Работы

1. Описание лаборатории

1.1 Структура проекта



1.2 Описание работы

Этот проект представляет собой систему управления распределением кофейной продукции, построенную с использованием **FastAPI** и интеграцией с базой данных PostgreSQL. Основная цель проекта — поддерживать управление учетными записями пользователей, разграничивать доступ и выполнять задачи, связанные с продуктами, заказами и клиентами.

Основная структура проекта

- Каталог app: Содержит логику приложения, основные модули:
 - database.py: Настройка подключения к базе данных с использованием SQLAlchemy.
 - models.py: Определение основных таблиц, таких как users и roles.

- main.py: Инициализация приложения и регистрация маршрутов.
- routers: Управление маршрутами:
 - auth.py: Регистрация, вход и аутентификация пользователей.
 - query.py: Выполнение SQL-запросов и запись логов.
- templates: HTML-шаблоны для регистрации, входа и панели управления.
- static: Содержит файлы CSS, JS и фоновые изображения.
- Файл requirements.txt: Список необходимых библиотек, включая:
 - FastAPI: Основной фреймворк.
 - SQLAlchemy: Управление базой данных.
 - Jinja2: Отрисовка HTML-шаблонов.
- Файл query.log: Логирование выполненных SQL-запросов.

Основные функции

- Управление учетными записями пользователей:
 - Регистрация и вход с использованием хэширования паролей.
 - Разграничение доступа в зависимости от роли (сотрудник или менеджер).
- Панель управления (Dashboard):
 - Отображение информации о пользователях.
 - Разграничение доступа и разрешённых операций в зависимости от роли.
- Выполнение SQL-запросов:
 - Интерфейс для выполнения запросов (для менеджеров).
 - Логирование запросов для последующей проверки.

Технологии и используемые языки

- Языки программирования:
 - Python (версия 3.10).
 - HTML, CSS и JavaScript (для пользовательского интерфейса).
- Основные фреймворки и библиотеки:
 - FastAPI: Быстрое и мощное создание API.
 - SQLAlchemy: Управление ORM и запросами к базе данных.
 - Jinja2: Генерация динамических HTML-страниц.
 - Werkzeug: Хэширование и проверка паролей.
- База данных:
 - PostgreSQL: Система управления реляционными базами данных.
- Дополнительные технологии:
 - AsyncIO: Асинхронная обработка для повышения производительности.
 - SessionMiddleware: Управление сессиями пользователей.

2. Выполнение программы

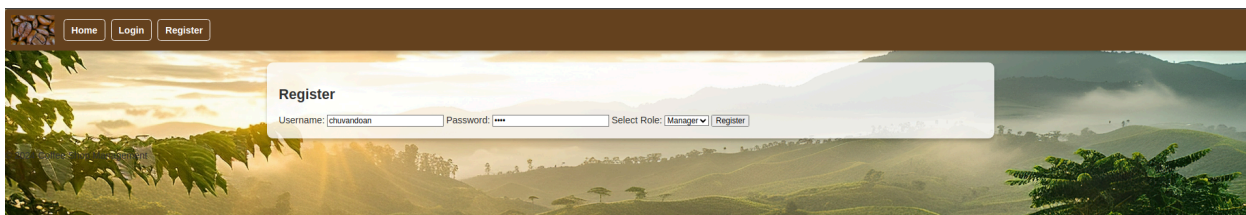


Рисунок 1 - Страница регистрации пользователя

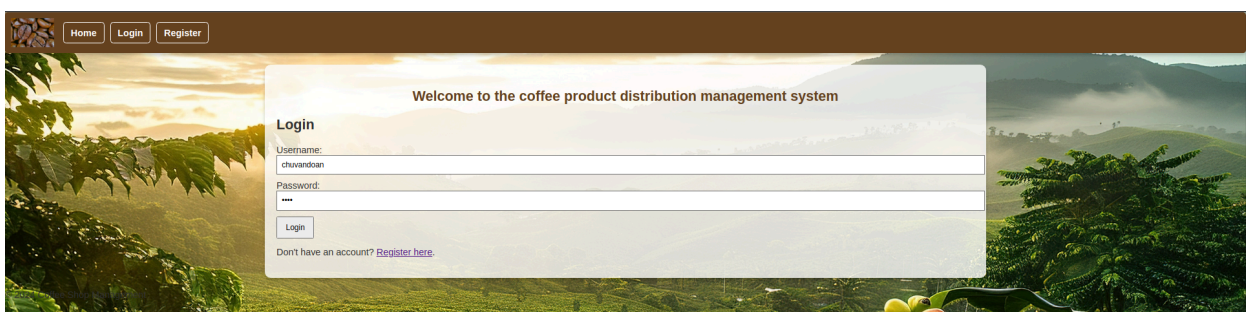


Рисунок 2 - Страница входа

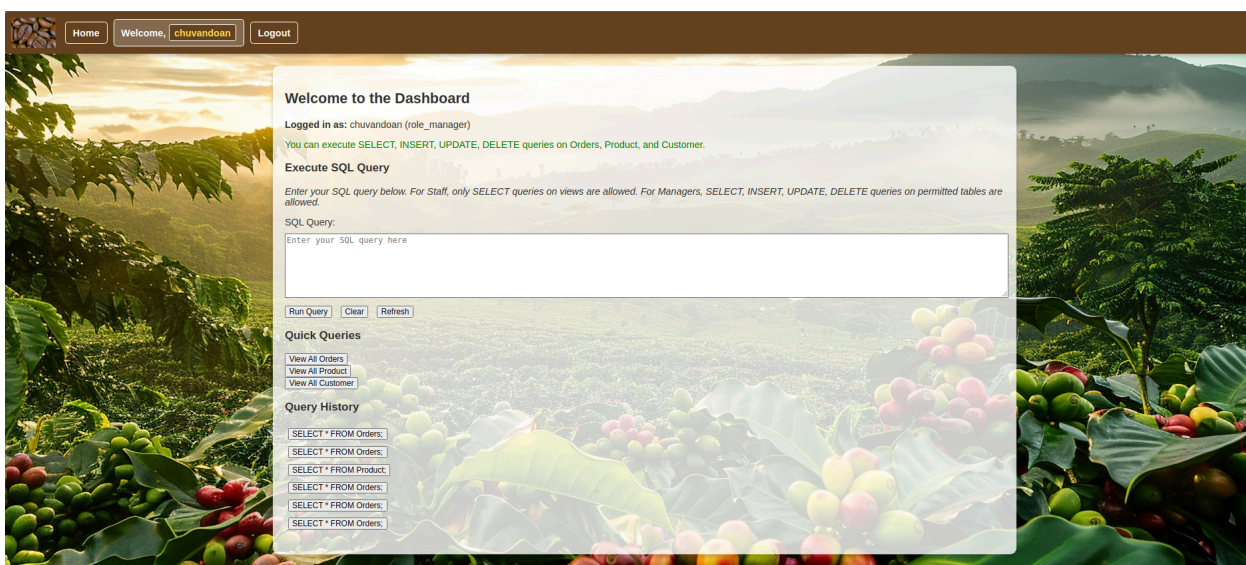


Рисунок 3 - Интерфейсная страница для взаимодействия с пользователем

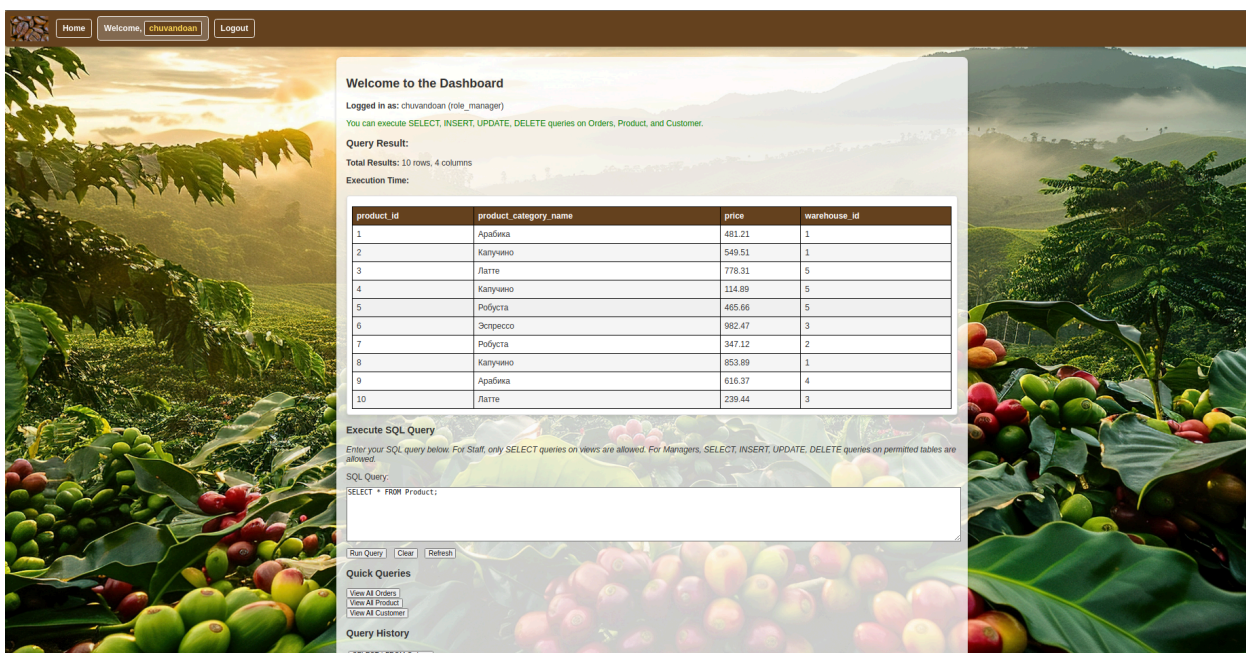


Рисунок 4 - Выполнение запроса в соответствии с ролью пользователя

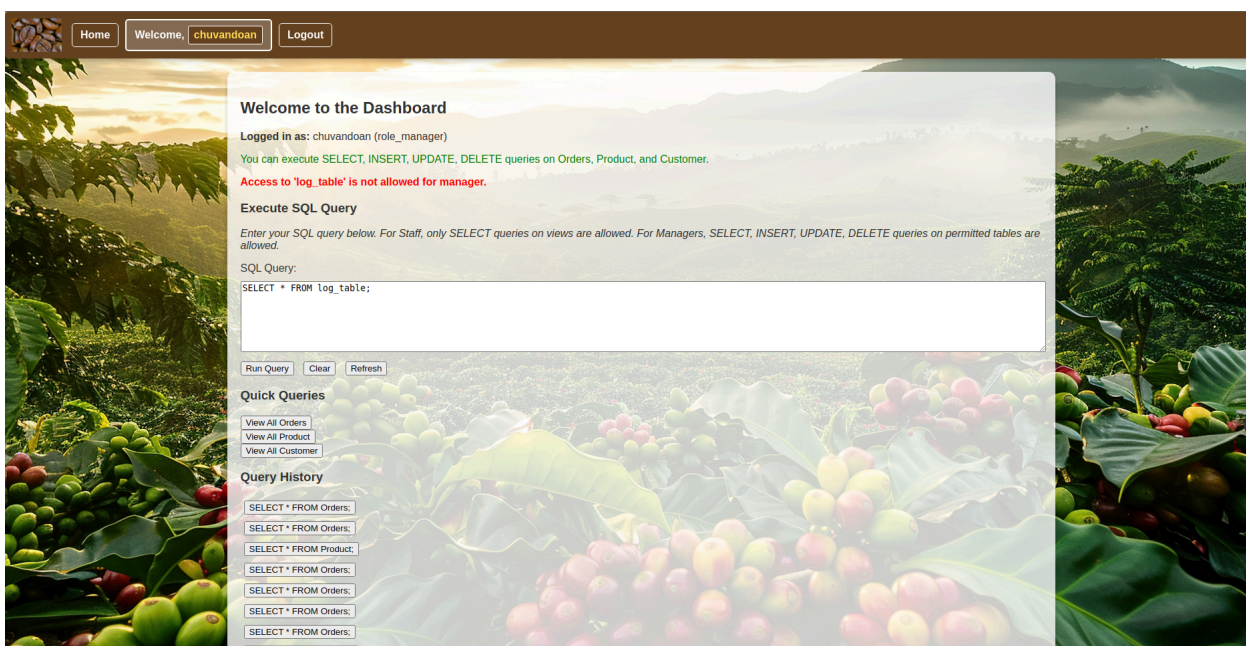


Рисунок 5 - Выполнение некорректного запроса в соответствии с ролью пользователя

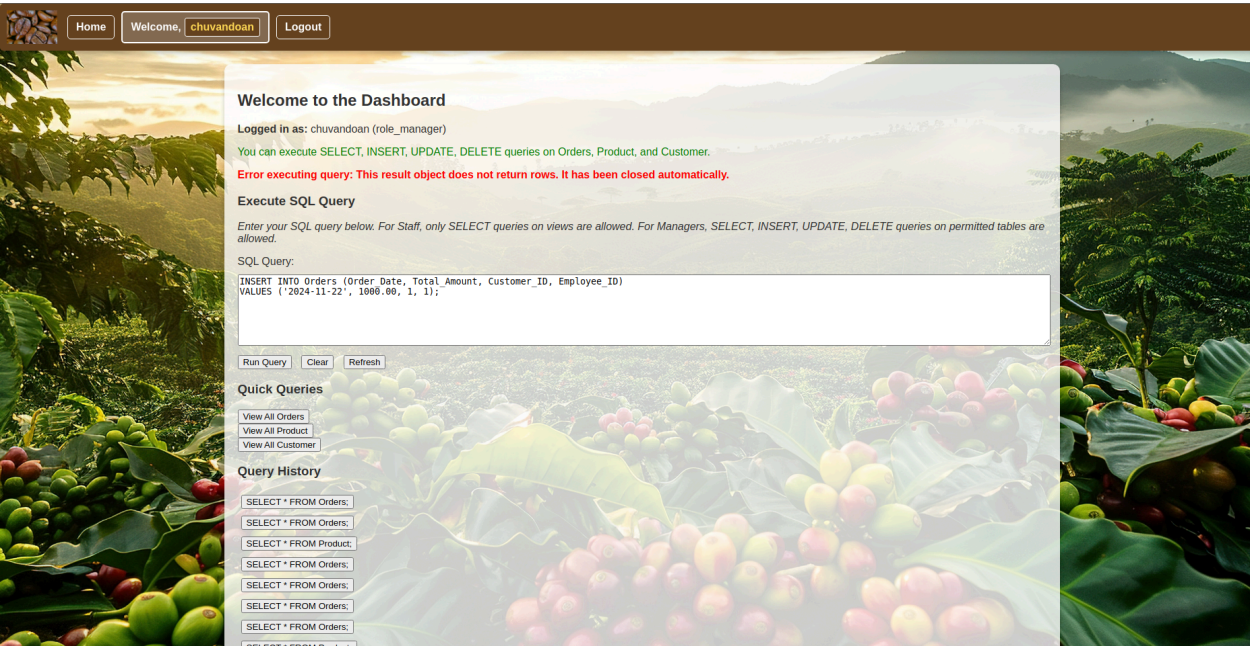


Рисунок 6 - Выполнение функции вставки (INSERT)

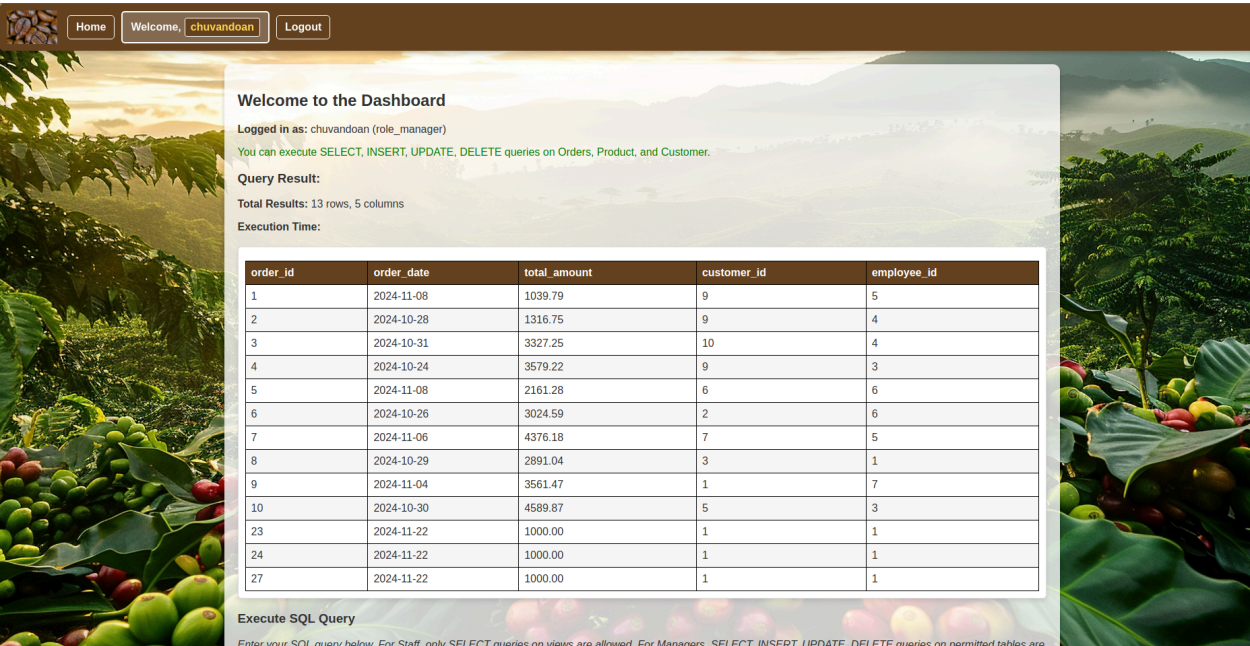


Рисунок 7 - Данные были добавлены

Data Output Messages Notifications						
≡ + 📄 ▼ 📋 ▼ 🗑️ 📦 ⬇️ 📈						
	order_id [PK] integer	order_date date	total_amount numeric (10,2)	customer_id integer	employee_id integer	
1	1	2024-11-08	1039.79	9	5	
2	2	2024-10-28	1316.75	9	4	
3	3	2024-10-31	3327.25	10	4	
4	4	2024-10-24	3579.22	9	3	
5	5	2024-11-08	2161.28	6	6	
6	6	2024-10-26	3024.59	2	6	
7	7	2024-11-06	4376.18	7	5	
8	8	2024-10-29	2891.04	3	1	
9	9	2024-11-04	3561.47	1	7	
10	10	2024-10-30	4589.87	5	3	
11	23	2024-11-22	1000.00	1	1	
12	24	2024-11-22	1000.00	1	1	
13	27	2024-11-22	1000.00	1	1	

Рисунок 8 - Проверка в PgAdmin 4

3. Основные моменты в разработке

3.1 Основной исходный код

- File main.py:

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from starlette.middleware.sessions import SessionMiddleware
from sqlalchemy.future import select
from app.database import engine, Base, get_db
from app.models import Role
from app.routers import auth, query

app = FastAPI()

app.add_middleware(SessionMiddleware, secret_key="chuvandoan")

templates = Jinja2Templates(directory="app/templates")

app.mount("/static", StaticFiles(directory="app/static"),
name="static")

@app.on_event("startup")
async def startup():
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
        result = await
conn.execute(select(Role).where(Role.name.in_(["role_manager",
"role_staff"])))
        existing_roles = [row.name for row in result.fetchall()]
        if "role_manager" not in existing_roles:
            await
conn.execute(Role.__table__.insert().values(name="role_manager"))
        if "role_staff" not in existing_roles:
            await
conn.execute(Role.__table__.insert().values(name="role_staff"))
        await conn.commit()

app.include_router(auth.router, prefix="/auth", tags=["auth"])
app.include_router(query.router, prefix="/query", tags=["query"])

@app.get("/", response_class=HTMLResponse)
async def read_root(request: Request):
```

```

        username = request.session.get("username")
        role = request.session.get("role")
        if not username or not role:
            return templates.TemplateResponse("base.html", {"request":
request, "error": "Please log in to continue."})
        return templates.TemplateResponse("base.html", {"request":
request, "username": username, "role": role})

@app.get("/healthcheck", response_class=HTMLResponse)
async def health_check():
    return {"status": "ok"}

@app.exception_handler(404)
async def not_found_handler(request: Request, exc):
    return templates.TemplateResponse("404.html", {"request":
request}, status_code=404)

```

- File database.py

```

from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "postgresql+asyncpg://postgres:0209@localhost/coffee_shop_db"

engine = create_async_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(bind=engine, class_=AsyncSession,
expire_on_commit=False)

Base = declarative_base()

async def get_db():
    async with SessionLocal() as session:
        yield session

```

- File models.py

```

from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from app.database import Base

class Role(Base):
    __tablename__ = "roles"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, unique=True, index=True)

```



```

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    username = Column(String, unique=True, index=True)
    password = Column(String, nullable=False)
    role_id = Column(Integer, ForeignKey("roles.id"))
    role = relationship("Role", back_populates="users")

Role.users = relationship("User", back_populates="role")

```

3.1 Маршруты (функциональные модули)

- routers/auth.py

```

from fastapi import APIRouter, Depends, HTTPException, Request, Form
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.templating import Jinja2Templates
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.future import select
from sqlalchemy import delete
from app.database import get_db
from app.models import User, Role
from werkzeug.security import generate_password_hash, check_password_hash

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")

@router.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    return templates.TemplateResponse("register.html", {"request":
request})

@router.post("/register")
async def register_user(
    request: Request,
    username: str = Form(...),
    password: str = Form(...),
    role: str = Form(...),
    db: AsyncSession = Depends(get_db),
):
    hashed_password = generate_password_hash(password)

```

```

        result = await db.execute(select(User).where(User.username ==
username))
    existing_user = result.scalar()
    if existing_user:
        return templates.TemplateResponse("register.html", {
            "request": request,
            "error": "Username already exists.",
        })
    result = await db.execute(select(Role).where(Role.name == role))
    role_obj = result.scalar()
    if not role_obj:
        return templates.TemplateResponse("register.html", {
            "request": request,
            "error": "Invalid role selected.",
        })
    new_user = User(username=username, password=hashed_password,
role_id=role_obj.id)
    db.add(new_user)
    await db.commit()
    return RedirectResponse(url="/auth/login", status_code=303)

@router.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    return templates.TemplateResponse("login.html", {"request": request})

@router.post("/login")
async def login_user(
    request: Request,
    username: str = Form(...),
    password: str = Form(...),
    db: AsyncSession = Depends(get_db),
):
    result = await db.execute(
        select(User, Role)
        .join(Role, User.role_id == Role.id)
        .where(User.username == username)
    )
    row = result.first()
    if not row:
        return templates.TemplateResponse("login.html", {
            "request": request,
            "error": "Invalid username or password.",
        })
    user, role = row
    if not check_password_hash(user.password, password):

```

```

        return templates.TemplateResponse("login.html", {
            "request": request,
            "error": "Invalid username or password.",
        })
    request.session["username"] = user.username
    request.session["role"] = role.name
    return RedirectResponse(url="/query", status_code=303)

@router.get("/users", response_class=HTMLResponse)
async def list_users(request: Request, db: AsyncSession = Depends(get_db)):
    username = request.session.get("username")
    role = request.session.get("role")
    if role != "role_manager":
        raise HTTPException(status_code=403, detail="Access forbidden: Managers only.")
    result = await db.execute(select(User).join(Role).order_by(User.id))
    users = result.fetchall()
    return templates.TemplateResponse("users.html", {
        "request": request,
        "users": users,
        "username": username,
        "role": role,
    })

@router.get("/logout")
async def logout(request: Request):
    request.session.clear()
    return RedirectResponse(url="/auth/login", status_code=303)

```

- routers/query.py

```

from fastapi import APIRouter, Request, Depends, Form
from fastapi.responses import HTMLResponse
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.sql import text
from app.database import get_db
from fastapi.templating import Jinja2Templates
import logging
import time

logging.basicConfig(filename="query.log", level=logging.INFO,
format="%asctime)s - %(message)s")

```

```

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")

query_history = []
ALLOWED_TABLES_MANAGER = ["orders", "product", "customer"]
ALLOWED_VIEWS_STAFF = ["sales_employee_view", "customer_view"]
FORBIDDEN_KEYWORDS = ["drop", "truncate", "alter"]

@router.get("/", response_class=HTMLResponse)
async def get_dashboard(request: Request):
    username = request.session.get("username")
    role = request.session.get("role")

    if not username or not role:
        return templates.TemplateResponse("login.html", {
            "request": request,
            "error": "Unauthorized. Please log in."
        })

    return templates.TemplateResponse("dashboard.html", {
        "request": request,
        "username": username,
        "role": role,
        "history": query_history,
        "result": None,
        "last_query": None,
        "error": None
    })

@router.post("/", response_class=HTMLResponse)
async def execute_query(
    request: Request,
    query: str = Form(...),
    db: AsyncSession = Depends(get_db)
):
    try:
        role = request.session.get("role")
        username = request.session.get("username")

        if not role or not username:
            return templates.TemplateResponse("dashboard.html", {
                "request": request,
                "error": "Unauthorized. Please log in.",
                "last_query": None,
                "history": query_history,

```

```

    })

    if any(keyword in query.lower() for keyword in FORBIDDEN_KEYWORDS):
        return templates.TemplateResponse("dashboard.html", {
            "request": request,
            "error": "Dangerous queries are not allowed.",
            "last_query": query,
            "history": query_history,
            "username": username,
            "role": role,
        })

    query_lower = query.strip().lower()
    query_parts = query_lower.split()
    command = query_parts[0]

    target_table_or_view = None
    if command in ["insert", "update", "delete"]:
        if command == "insert":
            target_table_or_view =
query_parts[query_parts.index("into") + 1]
        elif command == "update":
            target_table_or_view = query_parts[1]
        elif command == "delete" and "from" in query_parts:
            target_table_or_view =
query_parts[query_parts.index("from") + 1]
        elif "from" in query_parts:
            target_table_or_view = query_parts[query_parts.index("from") +
1]

    if target_table_or_view:
        target_table_or_view = target_table_or_view.rstrip(";").strip()

    if role == "role_manager":
        if command not in ["select", "insert", "update", "delete"]:
            return templates.TemplateResponse("dashboard.html", {
                "request": request,
                "error": f"Command '{command.upper()}' is not allowed
for manager.",
                "last_query": query,
                "history": query_history,
                "username": username,
                "role": role,
            })

```

```

        if target_table_or_view and target_table_or_view not in
[table.lower() for table in ALLOWED_TABLES_MANAGER]:
            return templates.TemplateResponse("dashboard.html", {
                "request": request,
                "error": f"Access to '{target_table_or_view}' is not
allowed for manager.",
                "last_query": query,
                "history": query_history,
                "username": username,
                "role": role,
            })
    elif role == "role_staff":
        if command != "select":
            return templates.TemplateResponse("dashboard.html", {
                "request": request,
                "error": "Only SELECT queries are allowed for staff.",
                "last_query": query,
                "history": query_history,
                "username": username,
                "role": role,
            })
        if target_table_or_view and target_table_or_view not in
[view.lower() for view in ALLOWED_VIEWS_STAFF]:
            return templates.TemplateResponse("dashboard.html", {
                "request": request,
                "error": f"Access to '{target_table_or_view}' is not
allowed for staff.",
                "last_query": query,
                "history": query_history,
                "username": username,
                "role": role,
            })

    logging.info(f"[{role.upper()}] {username} executed query:
{query}")

    result = await db.execute(text(query))
    if command in ["insert", "update", "delete"]:
        await db.commit()

    rows = result.fetchall() if command == "select" else []
    column_names = result.keys()

    data = [dict(zip(column_names, row)) for row in rows]

```



```

query_history.append(query)
if len(query_history) > 10:
    query_history.pop(0)

return templates.TemplateResponse("dashboard.html", {
    "request": request,
    "result": data,
    "last_query": query,
    "history": query_history,
    "total_rows": len(data),
    "username": username,
    "role": role,
})
except SQLAlchemyError as e:
    logging.error(f"Error executing query: {query} - {str(e)}")
    return templates.TemplateResponse("dashboard.html", {
        "request": request,
        "error": f"Error executing query: {str(e)}",
        "last_query": query,
        "history": query_history,
        "username": username,
        "role": role,
    })

```

4. Анализ характеристик приложения

Программа обладает четкой и хорошо организованной структурой, с модулями, разделенными по конкретным функциональным задачам. Это упрощает управление и сопровождение, особенно при необходимости расширения или добавления новых функций. Используемые технологии, такие как FastAPI, SQLAlchemy и Jinja2, обеспечивают высокую производительность и гибкость, что делает программу подходящей для современных систем управления. Благодаря асинхронной обработке приложение способно эффективно справляться даже с большим количеством запросов. Кроме того, программа включает базовые меры безопасности, такие как хэширование паролей и разграничение прав доступа пользователей, что гарантирует защиту данных. Простой и удобный интерфейс, а также поддержка логирования SQL-запросов являются дополнительным плюсом, позволяющим администраторам отслеживать и устранять ошибки по мере необходимости.

Тем не менее, в программе есть несколько недостатков, которые требуют доработки. Во-первых, отсутствуют интегрированные тесты, которые могли бы проверить стабильность и точность работы компонентов. Текущий интерфейс довольно простой, ему не хватает привлекательности, и он не оптимизирован для мобильных устройств. Кроме того, поддержка только базы данных PostgreSQL делает программу менее гибкой для интеграции с другими системами управления базами данных. Возможности масштабирования также ограничены, особенно при работе с крупными проектами или одновременно с несколькими сложными ролями.

Особое внимание стоит уделить безопасности: некоторые маршруты в приложении позволяют выполнять "сырые" SQL-запросы, что может привести к уязвимости перед атаками SQL-инъекций, если входные данные не проверяются должным образом. Управление сессиями (session) также реализовано недостаточно полно, отсутствуют функции, такие как запоминание учетной записи или расширенное управление сессиями. Кроме того, приложение не включает полноценную документацию API, что затрудняет его использование или дальнейшую разработку.

Вывод

Программа обладает хорошей структурой, использует современные технологии и обеспечивает базовый уровень безопасности. Она подходит для небольших проектов и простых систем управления. Однако для применения в крупномасштабных проектах необходимо доработать интерфейс, улучшить безопасность, добавить поддержку масштабирования, тестирование и полноценную документацию API.